

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ БУДІВНИЦТВА І
АРХІТЕКТУРИ

Кафедра Кібербезпеки та комп'ютерної інженерії

ЛАБОРАТОРНА РОБОТА № 5

з курсу “WEB-програмування”

на тему: “Реалізація інтерактивних компонентів веб-застосунку засобами фреймворку Vue.js 3 (Options API, JavaScript) без використання додаткових бібліотек, на основі підключення через CDN .”

Виконав

: група

бікс-23-1

Баранівський Андрій Валерійович

Перевірив

: Маленко М.

В.

Мета роботи:

Ознайомитися з концепцією компонентного підходу до побудови веб-інтерфейсів; набути практичних навичок роботи з фреймворком Vue.js 3 через Options API на мові JavaScript; освоїти реактивність даних, директиви шаблонів, обчислювані властивості (computed), спостерігачі (watch) та механізм передачі даних між компонентами через props та події (\$emit); навчитися підключати Vue.js через CDN без збірника.

Додаток 1 Завдання

12	Сайт кінотеатру	Розклад сеансів з фільтром за жанром	v-for, v-model, v-if, v-bind:class	computed для фільтрованих сеансів; компонент SessionCard з props та slot
----	-----------------	--------------------------------------	---	--

1. Файл index.html (Структура)

Цей файл визначає структуру вебсторінки кінотеатру та забезпечує інтеграцію з Vue.js.

- **Підключення Vue.js:**

Використовується Vue.js 3 через CDN (vue.global.js), що дозволяє працювати з реактивним інтерфейсом без додаткового збірника.

- **Директива v-cloak:**

Використовується разом зі стилем:

```
[v-cloak] {  
  display: none;  
}
```

щоб приховати необроблені Vue-шаблони до моменту повного завантаження застосунку.

- **Секція фільтрації сеансів (.filter-panel):**

- v-model="selectedGenre"

Забезпечує двосторонній зв'язок між списком <select> та реактивною змінною selectedGenre.

- <option value="...">

Дозволяє користувачу обирати жанр для фільтрації сеансів.

- **Секція розкладу сеансів (.sessions-container):**

- v-for="session in filteredSessions"

Динамічно створює картки сеансів на основі відфільтрованого списку.

- :key="session.id"

Встановлює унікальний ключ для кожного елемента списку, що оптимізує оновлення DOM.

- <session-card>

Користувачський Vue-компонент для відображення інформації про окремий

кіносеанс.

- `:class="{ active: selectedGenre === session.genre }"`

Динамічно додає CSS-клас для підсвічування карток активного жанру.

- **Умове відображення:**

- `v-if="filteredSessions.length === 0"`

Відображає повідомлення “Сеансів не знайдено”, якщо список після фільтрації порожній.

- **Slot у середині компонента:**

- `<template v-slot>`

Передає додатковий контент у компонент `SessionCard`, а саме інформацію про жанр фільму.

2. Файл `app.js` (Логіка додатка)

Центральний файл, який керує реактивними даними та логікою роботи сайту кінотеатру.

- **Метод `Vue.createApp()`:**

Створює екземпляр Vue-застосунку та монтує його до елемента `#app`.

- **`data()` — реактивні дані:**

- `selectedGenre`

Зберігає поточний обраний жанр для фільтрації.

- `sessions`

Масив об'єктів із даними про кіносеанси:

- назва фільму;
- жанр;
- час;
- зал;
- формат;
- ціна.

- **`computed` (обчислювані властивості):**

- `filteredSessions`

Автоматично повертає:

- або всі сеанси;
- або лише ті, що відповідають вибраному жанру.

Фільтрація виконується методом `filter()`.

- **`components`:**

- `SessionCard`

Реєструє дочірній компонент для використання всередині застосунку.

3. Файл `components/SessionCard.js` (Дочірній компонент)

Окремий Vue-компонент для відображення картки кіносеансу.

- **`props`:**

```
props: {  
  session: Object  
}
```

Компонент отримує дані про сеанс від батьківського компонента.

- **template:**

HTML-шаблон картки кіносеансу знаходиться всередині JS-файлу.

У шаблоні відображаються:

- назва фільму;
- час сеансу;
- номер залу;
- формат показу;
- ціна квитка.

- **slot:**

`<slot></slot>`

Дозволяє вставляти додатковий HTML-контент із батьківського компонента всередину картки.

- **Інтерполяція Vue:**

`{{ session.movie }}`

Використовується для динамічного виведення даних про сеанс.

4. Файл `styles.css` (Стилі)

Файл містить стилізацію всього сайту кінотеатру та забезпечує сучасний адаптивний дизайн.

- **CSS Variables (:root):**

Використовуються змінні для:

- кольорів;
- відступів;
- основної палітри сайту.

- **Стилізація `Header` та `Navigation`:**

Верхня частина сайту оформлена у темному стилі із золотими акцентами.

- **Flexbox для карток сеансів:**

`display: flex;`

`flex-wrap: wrap;`

`justify-content: center;`

Дозволяє адаптивно розташовувати картки на сторінці.

- **Стилізація `.session-card`:**

Для карток реалізовано:

- темний фон;
- заокруглення;
- тінь;
- `hover`-анімацію;
- плавні переходи.

- **Динамічний клас `.active`:**

Підсвічує картки активного жанру золотим кольором.

- **Стилізація `filter-panel`:**

Оформлює блок вибору жанру та список `<select>`.

- **Медіа-запити (`@media`):**

Забезпечують адаптивність сайту для планшетів та мобільних пристроїв:

- змінюється розташування меню;
- картки стають ширшими;
- елементи вирівнюються вертикально.

Додаток 1 (Код html)

```
<!DOCTYPE html>
<html lang="uk">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Кінотеатр Укр-сіті</title>
  <link rel="stylesheet" href="styles.css">
  <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
</head>
<body>

<div id="app">

  <header>
    <h1>Кінотеатр Укр-сіті</h1>
    <p>Кінотеатр нашої країни, аналогів якому нема!</p>
  </header>

  <nav>
    <ul>
      <li><a href="#">Фільми</a></li>
      <li><a href="#">Розклад</a></li>
      <li><a href="#">Про нас</a></li>
      <li><a href="#">Контакти</a></li>
    </ul>
  </nav>

  <main>

    <section>

      <h2 class="center-title">
        Розклад сеансів
      </h2>

      <div class="filter-panel">

        <label for="genre">
          Оберіть жанр:
        </label>

        <select id="genre" v-model="selectedGenre">

          <option value="all">
            Всі жанри
          </option>

          <option value="Фентезі">
            Фентезі
          </option>

          <option value="Екшен">
            Екшен
          </option>

          <option value="Фантастика">
            Фантастика
          </option>

          <option value="Драма">
```

```

        Драма
      </option>

    </select>

  </div>

  <p class="info">
    Знайдено сеансів:
    <strong>{{ filteredSessions.length }}</strong>
  </p>

  <div class="sessions-container">

    <session-card
      v-for="session in filteredSessions"
      :key="session.id"
      :session="session"
      :class="{ active: selectedGenre === session.genre }"
    >

      <template v-slot>
        <p class="slot-text">
          Жанр:
          {{ session.genre }}
        </p>
      </template>

    </session-card>

  </div>

  <p
    v-if="filteredSessions.length === 0"
    class="empty-message"
  >
    Сеансів не знайдено
  </p>

</section>

</main>

<footer>
  <p>Контакти: info@Ukr-city.ua</p>
  <p>© 2026 CinemaStar</p>
  <p>Гаряча лінія: 096 759 88 23</p>
</footer>

</div>

<script type="module" src="app.js"></script>

</body>
</html>

```

Код (styles.css)

```
:root {
  --color-primary: #0A0A0A;
  --color-secondary: #1a1a1a;
  --color-accent: #FFD700;
  --color-text: #f1f1f1;
  --color-gray: #bdbdbd;
  --color-card: #222;

  --spacing-sm: 10px;
  --spacing-md: 20px;
  --spacing-lg: 40px;
}

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: Arial, sans-serif;
  background: linear-gradient(180deg, #111, #1c1c1c);
  color: var(--color-text);
  line-height: 1.6;
}

header {
  background-color: var(--color-primary);
  padding: 30px 20px;
  text-align: center;
  border-bottom: 3px solid var(--color-accent);
}

header h1 {
  font-size: 3rem;
  color: var(--color-accent);
  margin-bottom: 10px;
}

header p {
```

```
    color: var(--color-gray);
    font-size: 1.2rem;
}
```

```
nav {
    background-color: var(--color-secondary);
    padding: 15px;
    border-bottom: 1px solid #333;
}
```

```
nav ul {
    list-style: none;
    display: flex;
    justify-content: center;
    gap: 35px;
}
```

```
nav a {
    color: var(--color-text);
    text-decoration: none;
    font-size: 1.1rem;
    transition: 0.3s;
}
```

```
nav a:hover {
    color: var(--color-accent);
}
```

```
main {
    max-width: 1200px;
    margin: auto;
    padding: 40px 20px;
}
```

```
.center-title {
    text-align: center;
    font-size: 2.5rem;
    margin-bottom: 30px;
    color: var(--color-accent);
}
```

```
.filter-panel {
    text-align: center;
    margin-bottom: 25px;
}
```

```
.filter-panel label {
```

```
    margin-right: 10px;
    font-size: 1.1rem;
}
```

```
.filter-panel select {
    padding: 10px 15px;
    border-radius: 8px;
    border: none;
    background-color: var(--color-card);
    color: white;
    font-size: 1rem;
}
```

```
.info {
    text-align: center;
    margin-bottom: 30px;
    font-size: 1.1rem;
}
```

```
.sessions-container {
    display: flex;
    flex-wrap: wrap;
    justify-content: center;
    gap: 25px;
}
```

```
.session-card {
    background-color: var(--color-card);
    width: 260px;
    padding: 20px;
    border-radius: 15px;
    border: 2px solid #333;
    transition: 0.3s;
    box-shadow: 0 0 10px rgba(0,0,0,0.5);
}
```

```
.session-card:hover {
    transform: translateY(-5px);
    border-color: var(--color-accent);
    box-shadow: 0 0 15px rgba(255,215,0,0.5);
}
```

```
.session-card h3 {
    color: var(--color-accent);
    margin-bottom: 15px;
    text-align: center;
}
```

```
.session-card p {  
  margin-bottom: 8px;  
}
```

```
.active {  
  border-color: var(--color-accent);  
  box-shadow: 0 0 15px rgba(255,215,0,0.7);  
}
```

```
.slot-text {  
  margin-top: 15px;  
  color: var(--color-accent);  
  font-weight: bold;  
  text-align: center;  
}
```

```
.empty-message {  
  text-align: center;  
  margin-top: 30px;  
  color: red;  
  font-size: 1.3rem;  
}
```

```
footer {  
  background-color: var(--color-primary);  
  text-align: center;  
  padding: 25px;  
  margin-top: 50px;  
  border-top: 3px solid var(--color-accent);  
}
```

```
footer p {  
  margin-bottom: 10px;  
  color: var(--color-gray);  
}
```

```
@media (max-width: 768px) {
```

```
  nav ul {  
    flex-direction: column;  
    align-items: center;  
  }
```

```
  header h1 {  
    font-size: 2rem;
```

```
}

.center-title {
  font-size: 2rem;
}

.sessions-container {
  flex-direction: column;
  align-items: center;
}

.session-card {
  width: 90%;
}
```

Код (app.js)

```
import SessionCard from './components/SessionCard.js';
```

```
const app = Vue.createApp({
  data() {
    return {
      selectedGenre: 'all',
      sessions: [
        {
          id: 1,
          movie: 'Аладін',
          genre: 'Фентезі',
          time: '12:00',
          hall: 'Зал 1',
          format: '3D',
          price: 150
        },
        {
          id: 2,
          movie: 'Месники: Фінал',
          genre: 'Екшен',
          time: '15:00',
          hall: 'Зал 2',
          format: 'IMAX',
          price: 180
        }
      ]
    }
  }
})
```

```

    },

    {
      id: 3,
      movie: 'Годзіла',
      genre: 'Фантастика',
      time: '18:00',
      hall: 'Зал 3',
      format: '2D',
      price: 200
    }
  ]
}

},

computed: {

  filteredSessions() {

    if (this.selectedGenre === 'all') {
      return this.sessions;
    }

    return this.sessions.filter(session =>
      session.genre === this.selectedGenre
    );

  }

},

components: {
  SessionCard
}

});

app.mount('#app');
```

Код (sessioncard.js)

```
export default {
```

```

props: {

  session: {
    type: Object,
    required: true
  }

},

template: `

<div class="session-card">

  <h3>{{ session.movie }}</h3>

  <p>Час: {{ session.time }}</p>

  <p>Зал: {{ session.hall }}</p>

  <p>Формат: {{ session.format }}</p>

  <p>Ціна: {{ session.price }} грн</p>

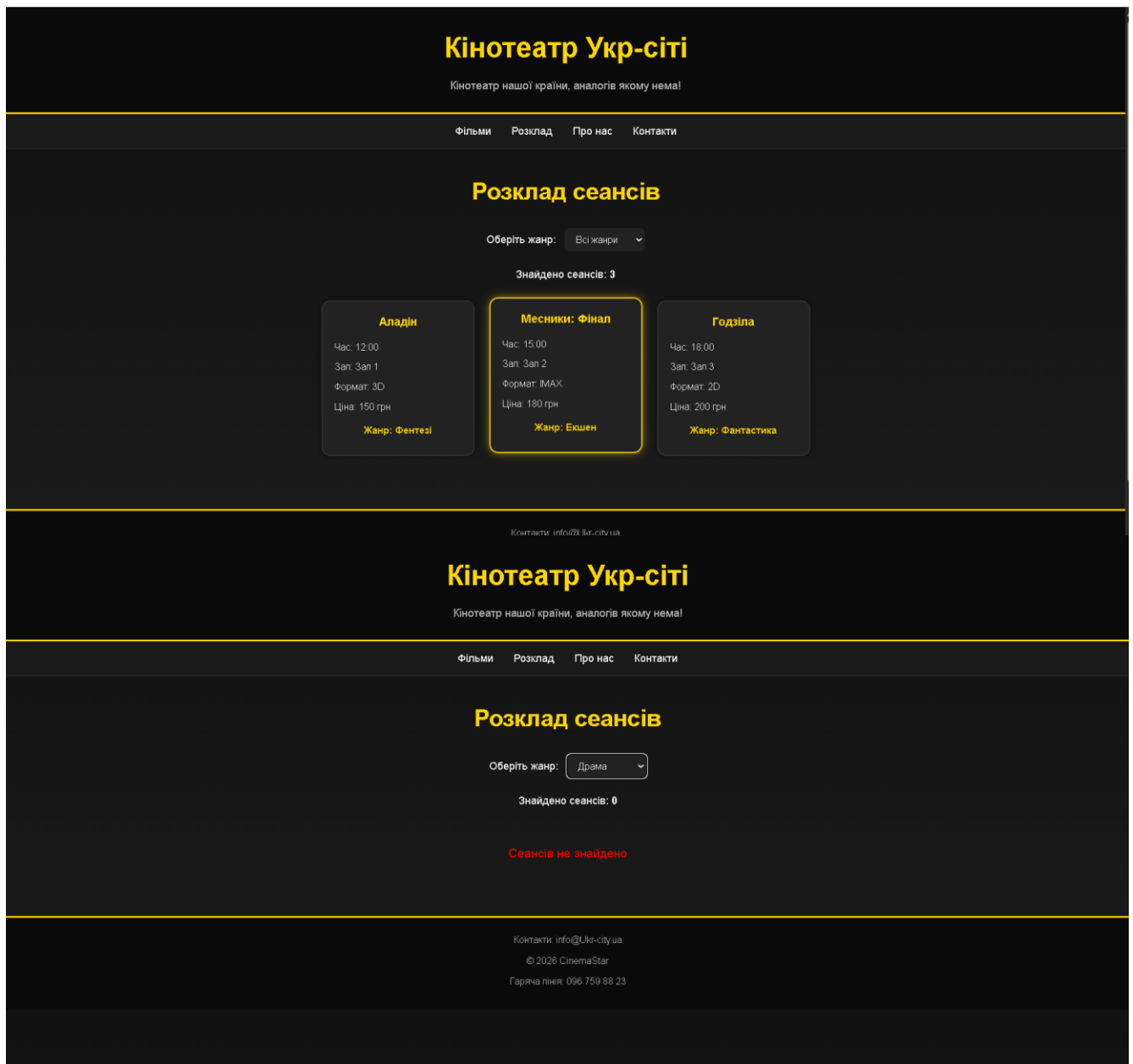
  <slot></slot>

</div>

`
}

```

Результат роботи:



Контрольні питання:

1. Реактивність у Vue.js

Реактивність — це здатність фреймворку автоматично оновлювати інтерфейс користувача (DOM) при зміні даних у стані додатка.

- Як Vue відстежує зміни: У Vue 3 це реалізовано за допомогою об'єктів `Proxy`. Коли ви створюєте стан (через **reactive** або **ref**), Vue обгортає ці дані проксі-об'єктом, який перехоплює операції читання та запису.
- Автоматичне оновлення: Коли властивість змінюється ("запис"), Vue сповіщає всіх "передплатників" (компоненти, які використовують ці дані), ініціюючи повторний рендеринг відповідної частини DOM.

2. Computed-властивості vs Методи

Хоча обидва підходи можуть повертати однаковий результат, вони

працюють по-різному:

Характеристика	Computed-властивість	Метод (Method)
Кешування	Кешується на основі реактивних залежностей. Оновлюється тільки при зміні вхідних даних.	Не кешується. Виконується заново при кожному зверненні або рендерингу.
Виклик	Використовується як властивість (без дужок: <code>fullName</code>).	Викликається як функція (з дужками: <code>fullName()</code>).
Коли використовувати	Для обчислень, що залежать від інших даних (фільтрація списків, підрахунок суми).	Для обробки подій (кліки) або виконання дій, що не залежать від стану.

3. Передача даних: props та \$emit

Цей механізм реалізує принцип "Data down, events up" (дані вниз, події вгору).

- **Props:** Використовуються для передачі даних від батьківського компонента до дочірнього.
- **\$emit:** Використовується дочірнім компонентом для відправки повідомлення (події) батькові про те, що щось сталося.
- Чому не можна змінювати props безпосередньо? У Vue діє односпрямований потік даних. Якщо дочірній компонент змінить пропс, це призведе до плутанини в стані додатка, оскільки батько про це не дізнається. Vue видасть попередження в консолі, щоб запобігти непередбачуваним помилкам.

4. Директива v-model

v-model забезпечує двостороннє зв'язування даних (two-way data binding) між кодом та елементами форми.

- "Під капотом": Це синтаксичний цукор для комбінації **v-bind** та **v-on**.
- Реалізація:
 - **v-bind:value="data"** — передає значення з коду в елемент (input).
 - **v-on:input="data = \$event.target.value"** — оновлює значення в коді при зміні тексту в полі.

5. Різниця між v-if та v-show

Обидві директиви керують видимістю, але з технічної точки зору вони різні:

- **v-if:** "Справжнє" умовне відображення. Якщо умова хибна, елемент повністю видаляється з DOM. Це економить ресурси пам'яті, але вимагає більше часу на перемикання.
- **v-show:** Завжди залишає елемент у DOM, але перемикає CSS-властивість **display: none**.
- Коли використовувати:
 - **v-show** — якщо елемент потрібно перемикати дуже часто.
 - **v-if** — якщо умова відображення рідко змінюється під час роботи.

6. Атрибут :key у директиві v-for

Атрибут **:key** допомагає віртуальному DOM (Virtual DOM) ідентифікувати кожен елемент у списку.

- Навіщо вказувати: Щоб Vue міг відстежувати ідентичність кожного вузла. При зміні списку (сортування, видалення) Vue не буде перемальовувати все заново, а просто перемістить існуючі вузли DOM.
- Що буде, якщо не вказати: Vue використає алгоритм "in-place patch".

Це може призвести до помилок у роботі станів компонентів, неправильної анімації або некоректного відображення вмісту полів вводу в списках. Використання індексу масиву як ключа вважається поганою практикою, якщо список може змінюватися

